

1-4 · Dockerfile 작성 및 이미지 최적화 · 정리·회고 0.5H

정리·회고

전후 수치를 나누고, 지시어 여덟 개를 한 장으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

30분 뒤에는

- ① 베이스 크기 차이와 재빌드 시간 차이를 옆 사람과 비교한다
- ② Dockerfile 지시어 여덟 개를 「언제 실행되나」로 정리한다
- ③ 자주 겪는 오류 세 가지의 원인과 해결을 안다
- ④ 1-5 Compose 에 쓸 app 폴더가 준비됐는지 확인한다

환경마다 값이 다르게 나와도 괜찮습니다 — 그 차이 자체가 이야깃거리

오늘 쟀 수치 두 가지 — 옆 사람과 비교

① 베이스 이미지 크기

01

python:3.12 와 python:3.12-slim 으로
빌드한 것의 SIZE 차이

② 재빌드 시간

02

좋은 순서와 나쁜 순서로 재빌드한
time 의 real 값 차이

`docker images data-app --format "{{.Tag}}\t{{.Size}}"` 로 한 번에 뽑아 주차 산출물 갤러리(패들릿)에 올려 두세요

Dockerfile 지시어 여덟 개 — 언제 실행되나

지시어	언제 실행되나	기억할 점
FROM	맨 처음	태그를 반드시 명시
WORKDIR	빌드 시점	없는 경로면 만들어 준다
COPY	빌드 시점	빌드 컨텍스트 안의 파일만
RUN	빌드 시점	굳는다 · 설치와 정리는 한 층에
ENV	빌드·실행 양쪽	이미지에 남는다
EXPOSE	문서화만	실제 포트는 docker run -p
CMD	컨테이너 시작 시	run 뒤에 명령을 주면 교체
ENTRYPOINT	컨테이너 시작 시	교체되지 않는 고정 명령

ENV 는 이미지에 남는다 — 비밀값은 넣지 않고 실행할 때 -e 나 시크릿으로

여덟 개를 스스로 확인해 보세요

- 1 FROM·WORKDIR·COPY·RUN·CMD 의 역할을 설명할 수 있다
- 2 data-app:0.1 을 빌드하고 실행해 출력을 확인했다
- 3 재빌드 로그에서 CACHED 층을 찾았다
- 4 나쁜 순서로 빌드해 재빌드 시간이 길어지는 것을 수치로 봤다
- 5 .dockerignore 를 만들어 빌드 컨텍스트를 줄였다
- 6 slim 과 전체 베이스의 크기를 docker images 로 비교했다
- 7 docker history 로 층별 크기를 들여다봤다
- 8 1-5 에서 쓸 app 폴더(Dockerfile 포함)가 완성돼 있다

증상으로 찾는 원인과 해결

증상	원인	해결
<code>"/requirements.txt"</code> <code>not found</code>	빌드 위치에 COPY 대상 파일이 없다	app 폴더로 cd 한 뒤 <code>docker build ...</code> 재실행
코드만 고쳤는데 매번 <code>pip install</code>	COPY .. 이 RUN <code>pip install</code> 보다 앞	<code>requirements</code> 먼저 → <code>pip install</code> → COPY ..
<code>can't open file</code> <code>'/app/app.py'</code>	WORKDIR 또는 COPY .. 이 빠짐	둘 다 넣고 재빌드 . <code>ls /app</code> 으로 파일 확인

마지막 점(.)이 빌드 컨텍스트 — `docker build` 는 `Dockerfile` 과 파일들이 있는 폴더에서 실행

자주 나오는 질문 네 가지

1 COPY 와 ADD 는 뭐가 다른가요?

ADD 는 압축 해제·URL 다운로드까지 해 동작 예측이 어렵다 — 공식 문서도 COPY 를 기본으로 권한다

2 캐시를 무시하고 처음부터 빌드하려면?

`docker build --no-cache` — 의존성이 이상하게 꼬였을 때

3 이미지에 비밀번호를 넣으면 안 되나요?

ENV 든 파일이든 층에 남아 `docker history` 로 보인다 — 실행할 때 `-e` 나 시크릿으로

4 태그를 안 붙이면?

`latest` 가 붙는다 — 1-3 의 `latest` 와 오늘의 `latest` 가 다른 이미지라 헷갈리니 버전 태그 습관

다음 회차 예고

1-5 · Docker Compose 이해하고 사용하기

01

이론 1H

Compose 개념과
YAML 구조

02

실습 1.5H

다중 컨테이너
구성 실습

03

실습 1H

주차 산출물:
통합 환경 구축

04

정리 0.5H

1주차 총정리
산출물 제출 확인

준비: 오늘 만든 app 폴더 그대로 · 1-3 의 pg 컨테이너가 떠 있으면 docker stop pg 로 5432 포트 비우기

1-4 · 정리·회고 0.5H 마무리

다음 회차는 1-5 Docker Compose

db·admin·app 세 서비스를 docker-compose.yml 하나로 묶어 1주차 산출물로 제출합니다

✓ **services·volumes·depends_on 구조**

✓ **docker compose up·ps·logs·down**

✓ **서비스 이름으로 컨테이너끼리 통신**

✓ **통합 환경 + README 제출**

준비할 것

app 폴더(Dockerfile 포함) · docker stop pg